HARDWARE DOCUMENTATION

ESP32 Vehicle Telemetry & Alert System

Wiring Notes, Component List and Circuit Walkthrough

B.Tech CSE (IoT), 3rd Year

29 June 2026

1. Project Overview

This board is a hand-wired ESP32 prototype with four communication paths on it: a CAN-bus interface for reading vehicle data, a 2.4GHz wireless link, a microSD module for logging, and a voice module for audio alerts. A DHT11 sensor is also on board for temperature and humidity. The ESP32 controls every module except the speaker, which the voice module drives on its own.

2. Components and Sensors Identified

Seven active components are on the board, not counting the ESP32 itself.

Component	Function	Interface Notes
ESP32 Development Module	Central microcontroller. Runs all control logic and hosts Wi-Fi / Bluetooth.	Hosts the shared SPI bus and every GPIO control line on the board.
nRF24L01 2.4GHz Wireless Transceiver	Wireless link for remote telemetry or control commands.	Standard 8-pin SPI device: CE, CSN, SCK, MOSI, MISO, IRQ, VCC, GND. The external SMA antenna points to a long-range (PA/LNA) variant.
MCP2515 CAN-Bus Module (onboard transceiver)	Reads frames from a vehicle CAN network.	SPI device with a dedicated interrupt line for frame-arrival signalling.
HW-125 Micro-SD Card Module	Local data logging.	SPI device, sharing the bus with the two items above.
ISD1820 Voice Record / Playback Module	Generates spoken or recorded audio alerts. Has a built-in microphone and three onboard buttons (REC, PLAYE, PLAYL).	Drives the speaker directly. Not yet wired to any ESP32 GPIO, so playback is presently manual-button only.
8Ω Dynamic Speaker	Audio output.	Wired directly to the ISD1820 only. Never connects to the ESP32.
DHT11 Temperature & Humidity Sensor	Cabin or ambient climate monitoring.	Single-wire digital data line, powered from 5V rather than 3.3V for steadier readings.

3. Power Architecture

Power is split across two rails, with one common ground shared by every module.

Rail	Source	Devices Powered
VIN (5V)	USB power input	DHT11 VCC only
3.3V	ESP32 onboard regulator	nRF24L01, MCP2515, HW-125, ISD1820

Rail	Source	Devices Powered
GND	Common ground loop	All seven components

Note: four 3.3V modules draw from the same onboard regulator that also powers the ESP32 itself, see Section 6 for why this is worth measuring before relying on it long-term.

4. ESP32 Connection Map

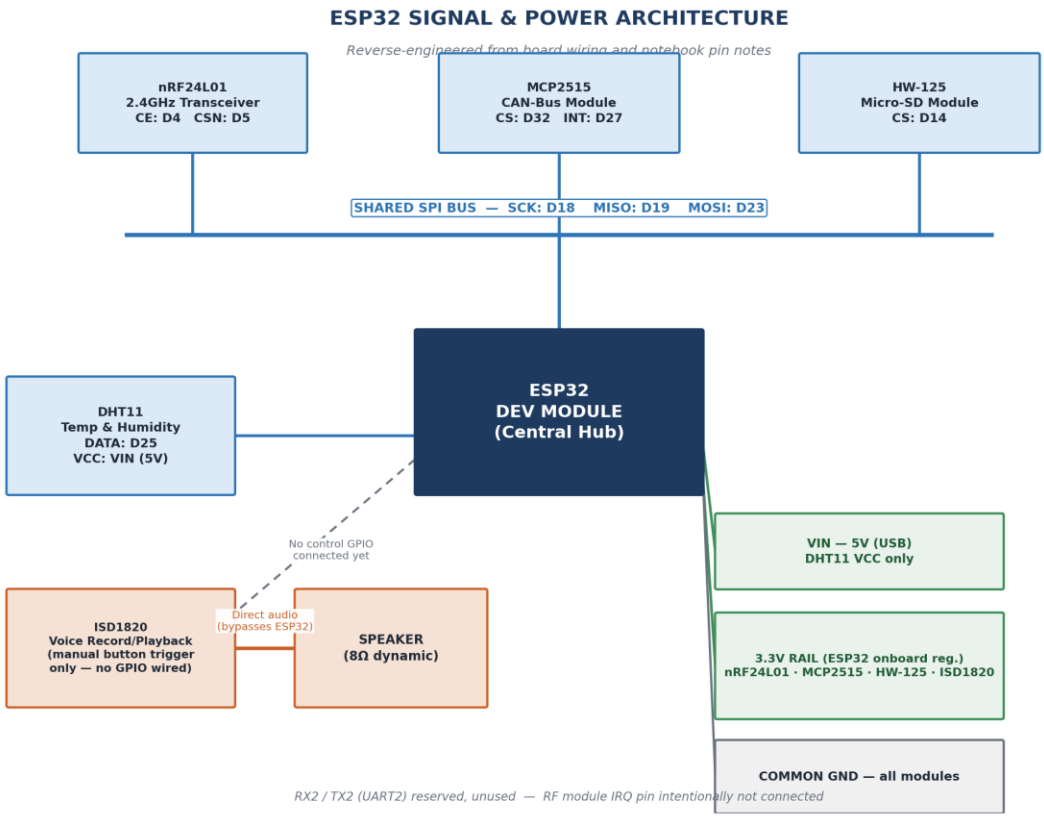
The table below is the complete, corrected pin map for the board.

ESP32 Pin	Connected Module(s)	Function	Notes
D4	nRF24L01	CE	Chip Enable, switches the module between standby and RX/TX.
D5	nRF24L01	CSN	SPI chip select for the wireless module.
D14	HW-125	CS	SPI chip select for the SD card module.
D18	nRF24L01, MCP2515, HW-125	SCK	Shared SPI clock for all three SPI devices.
D19	nRF24L01, MCP2515, HW-125	MISO	Shared SPI data line, module to ESP32.
D23	nRF24L01, MCP2515, HW-125	MOSI	Shared SPI data line, ESP32 to module.
D25	DHT11	DATA	Single-wire digital sensor read.
D27	MCP2515	INT	Assigned for interrupt-driven CAN frame reception. Confirm against the physical solder joint.
D32	MCP2515	CS	SPI chip select for the CAN module.
VIN	DHT11	VCC (5V)	Direct USB-rail power, red wire.
RX2 / TX2	Unused	UART2	Free for future expansion, e.g. GPS or a display.
nRF24L01 IRQ pin	Unused	—	Not wired. The link is polled rather than interrupt-driven.

Note: the nRF24L01's clock line is grouped with CAN and HW-125 on D18, matching the standard ESP32 SPI layout (SCK 18, MISO 19, MOSI 23). The earlier notebook entry pairing the transceiver with MOSI on the D18 line was a slip of the pen, not an actual second wire.

5. System Architecture

The diagram below shows how data and power actually move through the board. Colour in the diagram marks the type of connection: blue for the shared SPI data bus, orange for the direct audio path that bypasses the ESP32, and green for power.



One controller, one shared bus

The ESP32 is the only controller on the board. Three SPI peripherals, the nRF24L01, the MCP2515 CAN module and the HW-125 SD module, share one electrical bus (SCK, MISO, MOSI) and are told apart through their own chip-select lines (D5, D32, D14). This is the usual way to fit several SPI devices on a microcontroller without running three pins to each one.

CAN telemetry path

The MCP2515 listens for frames on the vehicle CAN network and raises its INT line (D27) the moment a frame arrives, so the ESP32 can react to it instead of constantly polling the bus. CAN_H and CAN_L are not yet wired to an external vehicle harness, so right now this is a bench-test addition and not a live vehicle connection.

Wireless link and logging path

The nRF24L01 gives a 2.4GHz wireless channel for sending status data out or receiving commands. The HW-125 writes sensor and telemetry data to a microSD card, so there is an offline log even if the wireless link drops.

Alert path, currently incomplete

The ISD1820 is wired straight to the speaker and never passes through the ESP32 for audio. Its PLAYE and REC trigger pins are not connected to any GPIO yet, so playback only happens if someone presses the onboard buttons by hand. Wiring one GPIO to PLAYE would let the firmware trigger a spoken alert on its own, for example after a CAN fault frame or a DHT11 reading crossing a set limit.

Power story

The DHT11 is the only component on the 5V rail, which is why its readings stay steady. Every other module pulls from the ESP32's own 3.3V regulator. That is convenient for wiring but worth checking once the CAN, RF and voice modules are all running at the same time.

6. Recommendations

These are practical next steps, not just things that would look good on paper. Most of them come from problems that tend to show up the first time a board like this actually gets used outside the lab.

- Wire the ISD1820's PLAYE pin to a free GPIO, D26 or D33 work fine since D27 is already used for the CAN interrupt. Right now someone has to physically press the button, which defeats the purpose if the goal is a real warning system, like a seatbelt reminder or an overheating alert that should fire on its own.
- Before powering the board again, check the D18 and D27 connections with a multimeter. Both were fixed on paper after re-reading the notes, not re-checked on the actual solder joints, and a wrong SPI or interrupt wire is a common way to get garbage readings or damage a module.
- The CAN module is not connected to a real vehicle yet. Adding an OBD-II breakout cable instead of hard-soldered wires would let this be plugged into different cars or a CAN bus simulator for testing, instead of being stuck to whatever it gets wired into first.
- Four modules pull 3.3V from the ESP32 board's own onboard regulator. These small regulators run warm under steady load, which is a known issue with cheap dev boards in general. Worth measuring the current draw with everything running together, and moving to a separate 3.3V regulator if it gets close to the limit.
- If the DHT11 breakout being used does not already have a pull-up resistor on the data line, add a 4.7k to 10k one. DHT11 readings are already not very precise, and a floating data line just adds more noise on top of that.
- The wiring is hard to follow as it is, with yellow, red, orange and black wires crossing all over the perfboard. Labelling each wire near the ESP32 end, or moving the SPI lines to small header connectors, would save a lot of time the next time anyone on the team has to debug or extend this.
- Range-test the nRF24L01 near or inside an actual car before assuming it will perform as listed. Online range numbers are usually measured in open air, and a car body is a lot of metal that will cut that range down by a noticeable amount.
- Put together some kind of basic enclosure, even a simple laser-cut or 3D-printed box, before this goes anywhere near a dashboard. An open perfboard with exposed solder joints is fine on a desk, but not safe near a car's metal body or anyone's hands.